

Copilot Studio connector setup for IsPilot

This guide prepares the integration between Microsoft Copilot Studio and the validated IsPilot API.

Scope

- Keep all business logic in the root IsPilot agent and the Cloud Run API layer.
- Use Copilot Studio only as the user experience and orchestration layer.
- Connect Copilot Studio to the production endpoint at:
<https://ispilot-api-46y2f3tyja-uc.a.run.app>

Recommended architecture

```
graph TD
    A[User in Teams / Copilot Studio] --> B[Custom Connector]
    B --> C[Cloud Run API]
    C --> D[Vertex AI Reasoning Engine]
    D --> E[IsPilot Coordinator + Specialist Agents]
```

API contract

Use the OpenAPI file in the API layer:

- [ispilot-api/openapi.yaml](#)

This file contains the working contract for:

- `GET /health`
- `POST /chat`
- bearer-token auth for production
- API-key auth for local testing

Production authentication flow

Use the following pattern to generate an identity token for the service account:

```
TOKEN=$(gcloud auth print-identity-token \
  --impersonate-service-account=sa-tot-osa@corp-stro-salesinventory-
  prod.iam.gserviceaccount.com)
```

Then call the API:

```
curl -sS -X POST "https://ispilot-api-46y2f3tyja-uc.a.run.app/chat" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer ${TOKEN}" \
--data '{"user_id":"debug-user","message":"How is Talca Colin performing?"}'
```

Connector configuration steps

1. Open Copilot Studio.
2. Create a new custom connector.
3. Import the OpenAPI specification from [ispilot-api/openapi.yaml](#).
4. Configure authentication:
 - Use OAuth 2.0 / Microsoft Entra ID or the service-account identity path approved by your environment.
 - Keep the Cloud Run API as the backend dependency.
5. Map the **POST /chat** action to the connector action.
6. Set the body schema to include:
 - **user_id**
 - **message**
 - **session_id** (optional)
7. Test the connector with a sample prompt:
 - "How is Talca Colin performing?"
8. Verify the response is returned in the connector UI without exposing the internal Agent Engine details.

Expected request payload

```
{
  "user_id": "sebastian",
  "message": "How is Talca Colin performing?",
  "session_id": null
}
```

Expected response payload

```
{
  "answer": "Talca Colín is performing at 95.16% SNSG with 0.67% Bodega and 4.17% Quiebre.",
  "session_id": "5411387247947677696",
  "request_id": "req_550e8400-e29b-41d4-a716-446655440000",
  "timestamp": "2026-08-29T03:38:06.750012",
  "status": "ok"
}
```

Validation checks

After connector setup, validate these flows:

- "How is Talca Colin performing?"
- "How can Talca Colin improve?"
- "What are today's priorities?"

The expected behavior is:

- Copilot Studio receives the user message.
- The custom connector calls the Cloud Run API.
- The Cloud Run API forwards the business question to the IsPilot reasoning engine.
- The agent responds with a business-oriented answer.

Important guardrails

- Do not move the specialist agent logic into Copilot Studio.
- Do not call Vertex AI directly from the custom connector.
- Keep the API as the authoritative integration layer.
- Treat the validated Cloud Run endpoint as the production contract.

Suggested follow-up

Next implementation steps:

1. Create the Copilot Studio custom connector.
2. Test the connector against the live Cloud Run URL.
3. Publish the connector inside the target Teams environment.
4. Add user-scoped audit fields and monitoring in the backend for production usage.